

B.C.A-I(Sem-II) 2013-14

Constructor and Destructor

Constructor:

- The constructor is special member function whose name is same as that of class name.
- The main task of constructor is to initialize values for object of its class.
- It is called constructor because it construct the values of data members of the class.
- The constructors are implicitly invoked by the compiler whenever object of its class is created.

Example:

Consider, there is one class having name “person” then its constructor is given as:

```
class    person
{
    public:
        person(); // constructor
};
```

Properties/Characteristics/Features of Constructor:

- 1) Constructor name is same as that of class name.
- 2) Constructors are only declared under public section of class.
- 3) Constructor does not have any return type even void.
- 4) Constructors are implicitly invoked by the compiler whenever object of class is created.
- 5) Constructor cannot be virtual.
- 6) Constructor can be overloaded.
- 7) They make implicitly call to the new operator when memory allocation is required.

Types of constructor:

Depending upon arguments passed to constructor, it has three types.

- 1) Default Constructor
- 2) Parameterized constructor
- 3) Copy constructor

1) Default constructor:

The constructor that does not have any argument or parameter as input, such constructor is called as “Default Constructor”.

Default constructor is implicitly invoked by compiler whenever object of class is crated

Syntax:

```
class    class_name
{
    public:
        class_name(); // default constructor.
};
```

Implementation of default constructor:

<pre>#include<iostream.h> #include<conio.h> class person { int age; char *name,*add; float sal; public: person(); // default constructor void show(); }; person::person() // defi. Of default constructor { age=70; name= "Sachin"; add= "Pune"; sal= 4568.50 }</pre>	<pre>void person::show() { cout<<"\n Age= "<<age; cout<<"\n Name= "<<name; cout<<"\n address= "<<add; cout<<"\n Salary= "<<sal; } void main() { person x; // call to def. constructor x.show(); getch(); }</pre>
---	---

2) **Parameterized constructor:**

- The constructor that accepts any argument or parameter as input, such constructor is called as “parameterized Constructor”.
- Parameterized constructor is implicitly invoked by compiler when we pass some values direct to object of class.

Syntax:

```
class    class_name
{
    public:
        class_name(datatype1,datatype2);    // parameterized constructor.
};
```

Implementation of parameterized constructor:

<pre>#include<iostream.h> #include<conio.h> class person { int age; char *name,*add; float sal; public: person(int,char*); // parameterized constructor void show(); }; person::person(int a, char *n) { age=a; name= n; add= "Pune"; sal= 4568.50 }</pre>	<pre>void person::show() { cout<<"\n Age= "<<age; cout<<"\n Name= "<<name; cout<<"\n address= "<<add; cout<<"\n Salary= "<<sal; } void main() { person x(45, "Rahul"); x.show(); getch(); }</pre>
---	--

3) **Copy constructor:**

- Copy constructor is used to create the new object of class.
- In case of copy constructor the reference of object is passed as argument to existing object, such that value or record hold by existing object is entirely copied into second object.
- Parameterized constructor is implicitly invoked by compiler when we pass reference of one object as an argument to another object of class.

Implementation of Copy constructor:

<pre>#include<iostream.h> #include<conio.h> class person { int age; char *name,*add; float sal; public: person(); person(person&); // copy constructor void show(); }; person::person() { age=20; name="Mahesh"; add="Pune"; sal=1845.50; }</pre>	<pre>person::person(person &x) //def. of copy constr. { age=x.age; name=x.name; add=x.add; sal=x.sal; } void person::show() { cout<<"\n Age= "<<age; cout<<"\n Name= "<<name; cout<<"\n Add= "<<add; cout<<"\n Sal= "<<sal; } void main() { person p; p.show(); person q(p); // call to copy constructor q.show(); getch(); }</pre>
---	---

Overloading of Constructor (Multiple Constructor in Class):

- We know that in case of function overloading, we can define or implement many functions having same name but these functions are differ from each other according to their signature (Type of argument and number of argument accept by function).
- Like function overloading, we are also able to overload the constructor.
- Constructor overloading means implementing or defining multiple constructor for single class. But all these constructors are differing from each other according to their signatures.

Implementation of Constructor overloading:

<pre>#include<iostream.h> #include<conio.h> class person { int age; char *name,*add; float sal; public: person(); person(int , char*); void show(); }; person::person() { age=70; name= "Sachin"; add= "Pune"; sal= 4568.50 }</pre>	<pre>person::person(int a, char *n) { age=a; name= n; add= "Pune"; sal= 4568.50 } void person::show() { cout<<"\n Age= "<<age; cout<<"\n Name= "<<name; cout<<"\n address= "<<add; cout<<"\n Salary= "<<sal; } void main() { person x; person y(45, "Rahul"); x.show(); y.show(); getch(); }</pre>
--	--

Constructor with default argument:

- We know that in case of default argument concept we specify some default value at the time of function declaration.
- Also, we can specify some default values in parameterized constructor. Then at the time of constructor invocation, if we don't pass argument then compiler by default assigns default values for formal parameters.

Following program shows constructor with default argument:

<pre>#include<iostream.h> #include<conio.h> class person { int age; char *name,*add; float sal; public: person(int=50,char[]= "RAHUL"); // constructor With def. argument void show(); }; person::person(int a, char n[]) { age=a; name= n; add= "Pune"; sal= 4568.50; }</pre>	<pre>void person::show() { cout<<"\n Age= "<<age; cout<<"\n Name= "<<name; cout<<"\n address= "<<add; cout<<"\n Salary= "<<sal; } void main() { person x; x.show(); getch(); }</pre>
---	---

Destructor:

- The destructor is also special member function whose name is same as that of class name but it precedes with (~) symbol.
- The main task of destructor is to destroy the object created by the constructor.
- Destructor destroys the object created by constructor means it releases or frees memory space for future use.
- The destructors are implicitly invoked by the compiler while we exit from program.

Example:

Consider, there is one class having name “person” then its destructor is given as:

```
class      person
{
    public:
        person();  // constructor
        ~ person(); // destructor

};
```

Properties/Characteristics/Features of Destructor:

- 1) Destructor name is same as that of class name but it precedes with tilde(~) sign
- 2) Destructor is only declared under public section of class.
- 3) Destructor does not have any return type even void.
- 4) Destructor does not accept any argument that's why they are not overloaded.
- 5) Destructor is implicitly invoked by the compiler while we exit from program.
- 6) Destructor can be virtual.
- 7) Destructor cannot be overloaded.

Note that:

- *The invocation i.e. execution of destructor is depends on number of objects for a class.*
- *Class having multiple constructors but it has only one destructor.*

Implementation of Destructor:

<pre>#include<iostream.h> #include<conio.h> class one { public: one(); // constructor ~one(); // destructor }; one::one() { cout<<"\n Constructor is called "; } one:: ~one() { cout<<"\n Destructor is called "; }</pre>	<pre>void main() { one x; one y; getch(); }</pre> OUT-PUT: Constructor is called Destructor is called <i>Note: After getting output press Alt+F5 keys then check output.</i>
--	--

Static data members and Static member functions:

➤ **Static data:**

- We know that, for single class we can create multiple objects. And each object holds their individual record. But there are some data which are common to all objects in such situation that data can be made as static.
- Once the data is made as static then this data is common to all objects of class.
- In short, static data is common data and which shared by all objects of class.
- Only one copy of static data members is maintained for entire class therefore it is shared among all objects.
- Static data members by default initialized to zero.
- Static data members are initialized at once therefore it is used like a counter.
- Static data member retain (keep) their values till at the end of program.
- Static data members are declared inside class body and defined outside the class because they are stored separately rather than as a part of an object.

- Syntax to declare static data member:

```
static    datatype    mem_name;
```

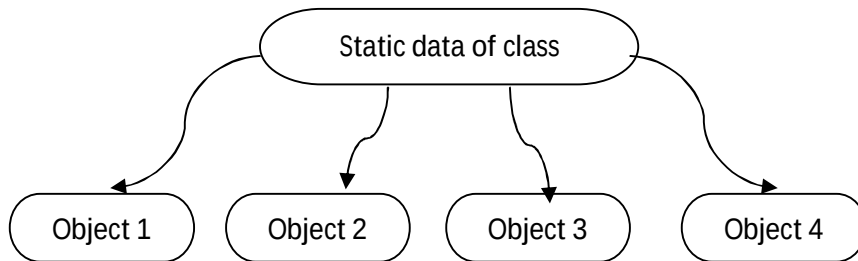
Here,

'static' is keyword which is used to declare member as static
 'datatype' is any valid data type in C++.
 'mem_name' is name of data member of class.

- Syntax to define static data member:

```
datatype    class_name :: mem_name=value;
```

Following fig. shows sharing of static data members by different objects of class:



In above fig. static data of class is common to all objects Object1, Object2, Object3, Object4. Or the same copy of static data is shared among all objects.

Following program demonstrate the use of static data members:

```
#include<iostream.h>
#include<conio.h>
class    student
{
    int roll;
    char name[20];
    float per;
    static char *cname;           //declaration of static data member
    public:
        void get( );
        void show( );
};
char*    student::cname="SANGOLA COLLEGE";    //def. of static data mem.
void student::get( )
{
    cout<<"\nEnter roll no: ";
    cin>>roll;
    cout<<"\nEnter Name: ";
    cin>>name;
    cout<<"\nEnter Percentage: ";
    cin>>per;
}
void student::show( )
{
    cout<<"\nRoll no= "<<roll;
    cout<<"\nName= "<<name;
    cout<<"\nPercentage="<<per;
    cout<<"\n College="<<cname;
}
void main( )
{
    clrscr();
    student a,b;
    a.get();
    b.get();
    a.show();
    b.show();
    getch();
}
```

➤ **Static member function:**

- The member functions that access only the static data members of class are called 'Static member function'.
- Static member function cannot access non static data directly i.e. Object of class is required to access non static data within static member function.
- Static member function access only static data directly.
- Static member functions are also common to all objects of class.
- The member functions can be made as 'static' by putting the keyword '*static*' before the function declaration.
- Static member function cannot declared with '*const*' keyword i.e. they are not constant.
- Static member function cannot virtual.
- Static member functions are accessed or invoked by using class name along with scope resolution operator. That is there is no need of object to call static member function.
- Following syntax shows the declaration of static member function.

```
static    return_type  mem_fun(argument list);
```

Here,

'static' is keyword which is used to declare member function as static

'return_type' is any valid data type in C++ that shows value return from function.

'mem_fun' is name of member function of class.

- Syntax for calling or accessing the static member function.

```
class_name :: mem_fun(arg1,arg2,...);
```

Here,

'class_name' is name of the class

'mem_fun' is name of static member function of class.

'arg1,arg2,...' are the arguments accept by the static member function.

Following program demonstrate the use of static data members and static member function to count total number of objects of class:

```
#include<iostream.h>
#include<conio.h>
class count
{
    static int cnt; //declaration of static data mem.
public:
    count();        //default constructor
    static void show(); //static mem. function
};
int count::cnt; //definition of static data member
count::count()
{
    cnt++;
}
void count::show()
{
    cout<<"\n Total objects= "<<cnt;
}
void main()
{
    clrscr();
    count a, b, c; //three times constructor invoked
    count::show(); // call to static mem. Fun.
    getch();
}
```

OUTPUT:

Total objects= 3

Assignment: 3

1. What is constructor? List out its characteristics
2. What is constructor? Explain types of constructor with one example.
3. Explain “Default argument constructor” with example.
4. What is destructor? List out its characteristics with one example.
5. Write short note on: “Constructor overloading”
6. Why destructor is not overloaded?
7. Differentiate between Constructor and Destructor.
8. What is static data member? List out its characteristics with one example.
9. What is static member function? List out its characteristics with one example.